

MOVIE TICKET BOOKING SYSTEM

Harshini Inta
2303114

REAL LIFE PROBLEM

You and your friend open BookMyShow at the same time

Both select Seat F-9 for Avengers

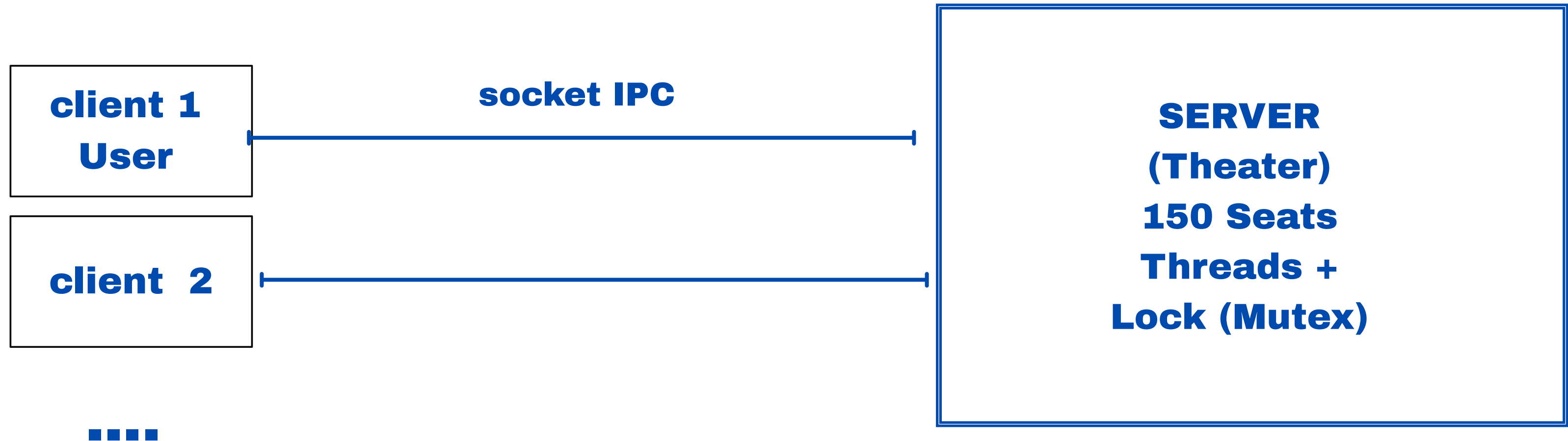
Both click "Pay Now"

?? Who gets the seat ??

Without proper OS concepts → DOUBLE BOOKING!
(2 people get same seat → chaos!)

My Solution (Architecture)

Client-Server Model (Just like real apps)



Features I Implemented :

- ✓ **Book single seat**
- ✓ **Book multiple seats together (e.g., for 4 friends)**
- ✓ **Cancel your own booking anytime**
- ✓ **Live seat status table (updates instantly)**
- ✓ **Works on multiple laptops (network ready)**
- ✓ **No double booking — 100% safe**

OS Concepts Demonstrated

- 1. Multithreading → Each user = one thread**
- 2. Synchronization (Mutex) → Only one can book at a time**
- 3. Critical Section → Code inside the lock**
- 4. Race Condition Prevention → Solved using mutex**
- 5. Deadlock Avoidance → Seats locked in increasing order**
- 6. Resource Sharing → All users share same seat array**
- 7. Concurrency → Many users work together**
- 8. Inter-Process Communication → Socket communication**
- 9. Atomicity → Book 4 seats = All or None**
- 10. Resource Release → Proper cancellation**

How Group Booking Works (Advanced!)

special Feature: Group Booking (4 seats)

User types: REQUEST 2 4 6 8

Server does:

1. Sort seats → 2,4,6,8

2. Check ALL are free

3. If yes → Book all 4 together

If no → Reject everything

→ Prevents partial booking

→ Prevents deadlock using resource ordering

Live Demo Time!

Conclusion & Learning

What I Learned

- **How real apps like BookMyShow prevent double booking**
- **Importance of threads, locks, and deadlock avoidance**
- **How operating systems manage shared resources safely**
- **Python sockets make IPC very easy**

**"The project implements a thread-safe,
socket-based movie ticket booking system demonstrating multithreading,
mutex synchronization,
critical section protection,
deadlock avoidance via resource ordering,
atomic multi-seat booking,
and inter-process communication
— accurately simulating real-world concurrent resource allocation scenarios."**